

IMAP Unit 2 Coding Reference Guide

Code level reference for Unit 2. Read it alongside the Unit 2 notes: the notes explain the concepts, this guide gives the complete code, commands and file layouts. It is not a replacement for the code files in each lab's demo folder.

1. Python Core

1.1 *args and **kwargs

```
# *args and **kwargs
def order_pizza(size, *toppings, **extras):
    print(size, toppings, extras)      # L ('olive', 'corn') {'cheese': True}

order_pizza("L", "olive", "corn", cheese=True)
order_pizza("L", *["olive", "corn"], **{"cheese": True})  # unpack at the call
```

1.2 Exception Handling

```
# exceptions
try:
    result = 10 / int(input("n: "))
except ZeroDivisionError:
    print("cannot divide by zero")
except ValueError as e:
    print("bad input:", e)
else:
    print("ok", result) # only if try had no error
finally:
    print("always runs")

raise ValueError("age cannot be negative")
# raise your own
```

1.3 Classes, Inheritance and Dunder Methods

```
# classes, inheritance, super, dunder methods
class Animal:
    species = "unknown"          # class attribute, shared by all instances

    def __init__(self, name):    # constructor
        self.name = name        # instance attribute

    def speak(self):
```

```

        return "..."
```

```

def __str__(self):                # print(obj)
    return f"{self.name}: {self.speak()}"

def __repr__(self):              # repr(obj), shell output
    return f"{type(self).__name__}({self.name!r})"
```

```

def __eq__(self, other):         # a == b
    return self.name == other.name

def __len__(self):              # len(obj)
    return len(self.name)
```

```

class Dog(Animal):
    def speak(self):            # override
        return "Woof"
```

```

class Puppy(Dog):
    def __init__(self, name, toy):
        super().__init__(name)  # run the parent constructor
        self.toy = toy

    def speak(self):
        return super().speak() + "!" # reuse the parent version
```

```

print(Dog("Rex"))               # Rex: Woof
print(Puppy("Bit", "ball"))     # Bit: Woof!
print(Dog("Rex") == Dog("Rex")) # True
```

Dunder method	Triggered by
<code>__init__</code>	<code>Dog(...)</code>
<code>__str__</code>	<code>print(obj)</code> , <code>str(obj)</code>
<code>__repr__</code>	<code>repr(obj)</code> , shell output
<code>__eq__</code>	<code>a == b</code>
<code>__len__</code>	<code>len(obj)</code>

1.4 The Four Pillars in Code

```

# the four pillars in a few lines
class Account:
    def __init__(self, balance):
```

```

    self._balance = balance          # encapsulation: internal by convention
    self.__pin = 1234                # name mangled to _Account__pin

    def deposit(self, amount):        # abstraction: callers do not see the checks
        if amount <= 0:
            raise ValueError("amount must be positive")
        self._balance += amount

for pet in [Dog("Rex"), Animal("Tom")]:
    print(pet.speak())                # polymorphism: same call, different result
# inheritance: class Dog(Animal) above

```

2. Networking and Files

2.1 Socket Client

Steps: `socket()`, `connect()`, `send()` / `recv()`, `close()`.

```

# socket client: raw HTTP request
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("example.com", 80))
s.sendall(b"GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n")
reply = b""
while chunk := s.recv(4096):        # empty bytes means the server closed
    reply += chunk
s.close()
print(reply.decode()[:300])

```

2.2 Socket Server (Echo)

Steps: `bind()`, `listen()`, `accept()`, `recv()` / `send()`, `close()`.

```

# socket server: echo (run in one terminal, connect from another)
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("127.0.0.1", 9000))
server.listen(5)
while True:
    conn, addr = server.accept()      # new socket per client
    data = conn.recv(1024)
    conn.send(b"Echo: " + data)       # sockets carry bytes, not str
    conn.close()

```

```
# client side of the echo test
# c = socket.socket(); c.connect(("127.0.0.1", 9000)); c.send(b"hi");
print(c.recv(1024).decode())
```

2.3 Files and JSON

```
# files: r read, w overwrite, a append, rb/wb binary
with open("notes.txt", "w") as f:
    f.write("line one\n")
with open("notes.txt", "a") as f:
    f.write("line two\n")
with open("notes.txt") as f:
    text = f.read()           # whole file
    # f.readline(), f.readlines(), or: for line in f:

# json: dumps/loads work on strings, dump/load on files
import json
order = {"id": 42, "items": ["samosa", "chai"], "paid": True}
text = json.dumps(order)
back = json.loads(text)
with open("order.json", "w") as f:
    json.dump(order, f)
with open("order.json") as f:
    data = json.load(f)
```

Mode	Behaviour
"r"	Read, file must exist
"w"	Write, overwrites
"a"	Append to the end
"r+"	Read and write
"rb", "wb"	Binary read, binary write

JSON function	Works on
json.dumps / json.loads	Strings
json.dump / json.load	Files

2.4 HTTP Request Format

```
GET /api/orders/42 HTTP/1.1
Host: example.com
```

Authorization: Token <token>

Method	Intent	Success status
GET	Read	200
POST	Create	201
PUT	Replace	200
DELETE	Remove	204

3. Django Project Setup

3.1 Setup Commands

```
python -m venv venv
source venv/bin/activate           # Windows: venv\Scripts\Activate.ps1
pip install django

django-admin startproject mysite .
python manage.py startapp orders
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver         # http://127.0.0.1:8000/
```

3.2 settings.py

```
# mysite/settings.py
INSTALLED_APPS = [
    # ... Django defaults ...
    "orders",                    # register every app you create
]
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "dashboard"
LOGOUT_REDIRECT_URL = "login"
SESSION_COOKIE_AGE = 60 * 60 * 24 * 7 # seconds
```

4. Django MVT

4.1 models.py

```
# orders/models.py
from django.db import models
from django.contrib.auth.models import User

class Order(models.Model):
    customer = models.ForeignKey(User, on_delete=models.CASCADE)
```

```

item = models.CharField(max_length=100)
quantity = models.IntegerField(default=1)
price = models.DecimalField(max_digits=8, decimal_places=2)
paid = models.BooleanField(default=False)
created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"{self.item} x {self.quantity}"

```

4.2 Migrations

```

python manage.py makemigrations      # write the migration file
python manage.py migrate              # apply it to the database

```

4.3 views.py

```

# orders/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import Order

def order_list(request):
    orders = Order.objects.filter(paid=False)
    return render(request, "orders/list.html", {"orders": orders})

def order_detail(request, pk):
    order = get_object_or_404(Order, pk=pk)
    return render(request, "orders/detail.html", {"order": order})

def order_create(request):
    if request.method == "POST":
        Order.objects.create(
            customer=request.user,
            item=request.POST["item"],
            quantity=int(request.POST["quantity"]),
            price=request.POST["price"],
        )
        return redirect("order-list")          # redirect after POST
    return render(request, "orders/form.html")

```

4.4 urls.py (app and project)

```

# orders/urls.py (create this file)
from django.urls import path
from . import views

urlpatterns = [

```

```

    path("", views.order_list, name="order-list"),
    path("new/", views.order_create, name="order-create"),
    path("<int:pk>/", views.order_detail, name="order-detail"),    # view(request,
pk=7)
]

# mysite/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("orders/", include("orders.urls")),
]

```

4.5 Templates

```

<!-- templates/base.html (add BASE_DIR / "templates" to TEMPLATES DIRS) -->
<!DOCTYPE html>
<html>
<body>
    <nav>Hi {{ request.user.username }}</nav>
    {% block content %}{% endblock %}
</body>
</html>

<!-- orders/templates/orders/list.html -->
{% extends "base.html" %}
{% block content %}
    {% if orders %}
        <p>{{ orders|length }} orders</p>
        {% for o in orders %}
            <p><a href="{% url 'order-detail' o.pk %}">{{ o.item }}</a>, {{
o.created_at|date:"d M Y" }}</p>
        {% endfor %}
    {% else %}
        <p>No orders yet.</p>
    {% endif %}
{% endblock %}

<!-- orders/templates/orders/form.html -->
{% extends "base.html" %}
{% block content %}
    <form method="post">

```

```

{% csrf_token %}
<input name="item" required>
<input name="quantity" type="number" value="1">
<input name="price" type="number" step="0.01" required>
<button type="submit">Save</button>
</form>
{% endblock %}

```

Template syntax	Purpose
<code>{{ variable }}</code>	Print a value
<code>{{ value length }}</code>	Filter (also <code>date</code> , <code>default</code> , <code>upper</code>)
<code>{% if %} {% else %} {% endif %}</code>	Branching
<code>{% for x in list %} {% endfor %}</code>	Loop
<code>{% url 'name' arg %}</code>	URL by name
<code>{% extends "base.html" %}</code>	Inherit the base layout
<code>{% block content %}{% endblock %}</code>	Fillable region
<code>{% csrf_token %}</code>	Required in every POST form

5. Auth and Sessions

5.1 accounts/views.py: Login, Logout, Register, Decorators, Sessions

```

# accounts/views.py
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User
from django.contrib.auth.decorators import login_required, permission_required
from django.shortcuts import render, redirect
from django.http import HttpResponse

def login_view(request):
    if request.method == "POST":
        user = authenticate(request, username=request.POST["username"],
                             password=request.POST["password"])
        if user is not None:                                # None means wrong credentials
            login(request, user)                             # creates the session and cookie
            return redirect("dashboard")
    return render(request, "index.html")

def logout_view(request):
    logout(request)                                         # clears the session
    return redirect("login")

```

```

def register_view(request):
    if request.method == "POST":
        user = User.objects.create_user(           # hashes the password
            username=request.POST["username"], password=request.POST["password"])
        login(request, user)
        return redirect("dashboard")
    return render(request, "register.html")

@login_required                                     # anonymous users go to
LOGIN_URL
def dashboard(request):
    return render(request, "dashboard.html")

@permission_required("orders.delete_order", raise_exception=True) # 403 without
it
def delete_order(request, pk):
    ...

def visits(request):                               # sessions in code
    request.session["n"] = request.session.get("n", 0) + 1
    return HttpResponse(request.session["n"])

```

5.2 accounts/urls.py

```

# accounts/urls.py, included from mysite/urls.py as path("accounts/",
include("accounts.urls"))
from django.urls import path
from . import views

urlpatterns = [
    path("login/", views.login_view, name="login"),
    path("logout/", views.logout_view, name="logout"),
    path("register/", views.register_view, name="register"),
    path("dashboard/", views.dashboard, name="dashboard"),
]

```

5.3 Templates: CSRF and Login Checks

```

<!-- index.html : every POST form needs the csrf tag -->
<form method="post">
    {% csrf_token %}
    <input name="username">
    <input name="password" type="password">
    <button type="submit">Log in</button>

```

```
</form>
```

```
<!-- checks in templates -->
```

```
{% if request.user.is_authenticated %}Welcome {{ request.user.username }}{% endif %}
```

```
{% if perms.orders.delete_order %}<button>Delete</button>{% endif %}
```

5.4 Permission Checks and Groups

```
# checks in code, and groups (python manage.py shell)
```

```
request.user.is_authenticated
```

```
request.user.has_perm("orders.delete_order")      # add_, change_, delete_, view_  
per model
```

```
from django.contrib.auth.models import Group, Permission
```

```
editors = Group.objects.create(name="editors")
```

```
editors.permissions.add(Permission.objects.get(codename="delete_order"))
```

```
User.objects.get(username="asha").groups.add(editors)
```

6. Database Connectivity

6.1 PostgreSQL in Docker

```
pip install psycopg2-binary
```

```
docker run --name imap-postgres -e POSTGRES_PASSWORD=yourpassword \
```

```
-e POSTGRES_DB=imapdb -p 5432:5432 -d postgres:16
```

```
docker start imap-postgres      # next time
```

6.2 DATABASES Setting

```
# mysite/settings.py
```

```
DATABASES = {
```

```
    "default": {
```

```
        "ENGINE": "django.db.backends.postgresql",
```

```
        "NAME": "imapdb",
```

```
        "USER": "postgres",
```

```
        "PASSWORD": "yourpassword",      # in real projects:
```

```
os.environ["DB_PASSWORD"]
```

```
        "HOST": "localhost",
```

```
        "PORT": "5432",
```

```
    }
```

```
}
```

```
# then: python manage.py migrate    and    python manage.py createsuperuser
```

6.3 ORM Queries

```
# ORM
from django.db.models import Q, Sum, Count

Order.objects.all()
Order.objects.get(pk=7) # exactly one, else an error
Order.objects.filter(paid=False, price__gte=50) # AND
Order.objects.filter(Q(item="tea") | Q(item="chai")) # OR
Order.objects.exclude(paid=True)
Order.objects.filter(item__icontains="te") # also __startswith, __in,
__lt, __range
Order.objects.order_by("-price")[:5] # top 5 by price
Order.objects.filter(paid=False).count()
Order.objects.aggregate(total=Sum("price"))
Order.objects.create(customer=u, item="tea", price="40")
Order.objects.filter(paid=False).update(paid=True)
o.save(); o.delete()
print(Order.objects.filter(paid=False).query) # the SQL Django writes
```

Lookup	Meaning
<code>__gt</code> , <code>__gte</code> , <code>__lt</code> , <code>__lte</code>	Greater, less (or equal)
<code>__contains</code> , <code>__icontains</code>	Text contains (case insensitive)
<code>__startswith</code>	Text starts with
<code>__in=[...]</code>	Value in a list
<code>__range=(a, b)</code>	Between two values

6.4 Raw SQL

```
# raw SQL: always placeholders, never string concatenation
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute("SELECT username FROM auth_user WHERE id = %s", [user_id])
    row = cursor.fetchone()
```

7. Django REST Framework

7.1 Install

```
pip install djangorestframework
```

7.2 settings.py

```
# mysite/settings.py
INSTALLED_APPS += ["rest_framework", "rest_framework.authtoken"]
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.TokenAuthentication",
        "rest_framework.authentication.SessionAuthentication",
    ],
}
```

7.3 Serializer and ViewSet

```
# orders/serializers.py
from rest_framework import serializers
from django.contrib.auth.models import User

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ["id", "username", "email"] # password never exposed

# orders/api_views.py
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated
from django.contrib.auth.models import User
from .serializers import UserSerializer

class UserViewSet(viewsets.ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [IsAuthenticated] # default is AllowAny
```

7.4 Router and URLs

```
# mysite/urls.py
from rest_framework.routers import DefaultRouter
from orders.api_views import UserViewSet

router = DefaultRouter()
router.register("users", UserViewSet) # /api/users/ and
/api/users/<id>/

urlpatterns += [path("api/", include(router.urls))]
```

7.5 Token and Testing

```
python manage.py migrate # creates the token table
python manage.py drf_create_token asha # prints the token

curl http://127.0.0.1:8000/api/users/ -H "Authorization: Token <token>" # 200
curl -i http://127.0.0.1:8000/api/users/ # 401
```

Status	Meaning
200, 201, 204	OK, created, deleted
400	Serializer validation failed
401	Missing or wrong token
403	Logged in but not allowed
404	Object not found

7.6 Permission Classes

Class	Allows
AllowAny	Everyone (the default)
IsAuthenticated	Valid token or session
IsAdminUser	is_staff = True only
IsAuthenticatedOrReadOnly	Anyone reads, logged in users write

8. Machine Learning Model Integration

8.1 Install and Create the App

```
pip install scikit-learn joblib
python manage.py startapp mlpredict_iris
mkdir mlpredict_iris/ml
```

8.2 Train and Save the Model

```
# train_model.py (run once: python train_model.py)
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import joblib

X, y = load_iris(return_X_y=True)
clf = RandomForestClassifier().fit(X, y)
joblib.dump(clf, "mlpredict_iris/ml/iris_model.pkl")
```

8.3 Load the Model Once

```
# mlpredict_iris/ml_model.py : loaded once at import, not per request
import joblib
from pathlib import Path

MODEL_PATH = Path(__file__).resolve().parent / "ml" / "iris_model.pkl"
model = joblib.load(MODEL_PATH)
```

8.4 Serializer, View and URLs

```
# mlpredict_iris/views.py
from rest_framework import serializers
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from .ml_model import model

class IrisInputSerializer(serializers.Serializer):
    sepal_length = serializers.FloatField()
    sepal_width = serializers.FloatField()
    petal_length = serializers.FloatField()
    petal_width = serializers.FloatField()

class PredictView(APIView):
    permission_classes = [IsAuthenticated] # without it the model is open to all

    def post(self, request):
        s = IrisInputSerializer(data=request.data)
        s.is_valid(raise_exception=True) # bad input returns 400
        d = s.validated_data
        p = model.predict([[d["sepal_length"], d["sepal_width"],
                           d["petal_length"], d["petal_width"]]])[0] # List
inside a list
        return Response({"prediction": int(p)}) # int(): NumPy ints are not JSON safe

# mlpredict_iris/urls.py : path("api/predict/", PredictView.as_view())
# mysite/urls.py : path("", include("mlpredict_iris.urls"))
```

8.5 Test With curl

```
curl -X POST http://127.0.0.1:8000/api/predict/ \
-H "Authorization: Token <token>" -H "Content-Type: application/json" \
-d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
# {"prediction": 0}
```

8.6 Call It From a Browser Page

```
// from a logged in browser page: session cookie plus CSRF header, no token
function getCookie(name) {
  return document.cookie.split("; ")
    .find(row => row.startsWith(name + "="))?.split("=")[1];
}
const res = await fetch("/api/predict/", {
  method: "POST",
  headers: { "Content-Type": "application/json", "X-CSRFToken":
getCookie("csrftoken") },
  body: JSON.stringify({ sepal_length: 5.1, sepal_width: 3.5, petal_length: 1.4,
petal_width: 0.2 }),
});
console.log(await res.json());
```

9. Common Errors and Fixes

Error	Fix
<code>NameError</code> in <code>urls.py</code> or <code>views.py</code>	Add the missing import (<code>path</code> , <code>include</code> , <code>views</code>)
Model or template not found	Add the app to <code>INSTALLED_APPS</code>
<code>no such table</code> or <code>relation does not exist</code>	<code>python manage.py migrate</code>
Model change not in database	Run both <code>makemigrations</code> and <code>migrate</code>
<code>TemplateDoesNotExist</code>	Path must be <code>app/templates/app/page.html</code>
<code>NoReverseMatch</code>	<code>{% url %}</code> name must match <code>name=</code> in <code>urls.py</code> , pass <code>o.pk</code> if the path has <code><int:pk></code>
<code>MultiValueDictKeyError</code>	Form <code>name=</code> must match <code>request.POST["x"]</code>
403 CSRF verification failed	Add <code>{% csrf_token %}</code> or send <code>X-CSRFToken</code>
User cannot log in after creation	Use <code>create_user</code> , not <code>User(...)</code>
<code>connection refused</code> on 5432	<code>docker start imap-postgres</code>
<code>ModuleNotFoundError</code>	<code>pip install</code> the package, check the venv is active
API 401	Header must be <code>Authorization: Token <key></code>
API open to everyone	Set <code>permission_classes</code> , default is <code>AllowAny</code>
<code>int64</code> is not JSON serializable	Wrap in <code>int(p)</code>
<code>Expected 2D array</code>	<code>model.predict([[a, b, c, d]])</code>
<code>a bytes-like object is required</code>	<code>.encode()</code> before <code>send</code> , <code>.decode()</code> after <code>recv</code>

10. Command Sheet

```
python -m venv venv # virtual environment
pip install django djangorestframework psycopg2-binary scikit-learn joblib
django-admin startproject mysite .
python manage.py startapp <app>
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
python manage.py shell
python manage.py drf_create_token <username>
docker run --name imap-postgres -e POSTGRES_PASSWORD=... -p 5432:5432 -d
postgres:16
docker start imap-postgres
```